

PRODUCT REQUIREMENTS DOCUMENT

CredVault

Credit Card Management Platform — Backend MVP

Single-Developer Rescoped Edition

Document Version	2.0.0 (MVP Rescope)
Original Version	1.0.0 — Enterprise Edition (5-Member Team)
Date	August 2026
Team Size	1 Developer
Estimated Duration	10 – 12 Days
Status	Draft — Approved for Development
Primary Tech Stack	ASP.NET Core (.NET 10) · EF Core 10 · SQL Server
Domain	Fintech / Credit Management

CONFIDENTIAL — FOR INTERNAL USE ONLY

Version History

Version	Date	Author	Description
1.0.0	March 2026	Team Lead (Architect)	Original enterprise-scale PRD authored for a 5-member team: full microservices architecture (11 services), API Gateway, CQRS with MediatR, RabbitMQ event-driven messaging, Saga orchestration, Angular 19 SPA, Docker + CI/CD.
2.0.0	August 2026	Solution Architect	Rescoped for a single developer working within a 10–12 day timeline. Reduced to two backend services (Identity, Credit Management) covering Card, Bill, and Payment management. Advanced enterprise patterns (CQRS, Saga, RabbitMQ, microservice fan-out) preserved conceptually and moved to Future Enhancements. Vision, domain, and terminology of the original CredVault platform retained throughout.

This document supersedes v1.0.0 as the active build specification. v1.0.0 remains the long-term architectural vision that the MVP is designed to grow into.

Table of Contents

Version History.....	2
Table of Contents.....	3
1. Executive Summary.....	5
2. Project Vision	6
3. Problem Statement.....	7
4. Project Scope.....	8
4.1 In Scope (MVP)	8
4.2 Out of Scope for MVP (see Section 22 — Future Enhancements)	8
5. Functional Scope	9
6. Technology Stack	10
7. High-Level Architecture	11
8. Microservice Overview	12
8.1 Identity Service	12
8.2 Credit Management Service	12
9. Clean Architecture	13
10. Repository Pattern.....	14
11. Service Pattern	15
12. Folder Structure	16
13. Database Design.....	17
13.1 IdentityServiceDb.....	17
Roles	17
Users.....	17
13.2 CreditManagementDb	17
Cards.....	17
Bills	18
Payments.....	18
14. Entity Relationship Diagram	19
14.1 IdentityServiceDb.....	19
14.2 CreditManagementDb	19
15. Functional Requirements.....	20
15.1 Identity Module	20
15.2 Card Module.....	20
15.3 Bill Module	20
15.4 Payment Module.....	21
15.5 Optional Enhancement — Overview.....	21
16. API Design.....	22

16.1 Authentication & Users — Identity Service	22
16.2 Cards — Credit Management Service.....	22
16.3 Bills — Credit Management Service	22
16.4 Payments — Credit Management Service.....	23
16.5 Optional Enhancement — Overview	23
17. Authentication Flow	24
18. Validation Strategy	25
19. Exception Handling Strategy.....	26
20. Logging Strategy.....	27
21. Development Timeline (10–12 Days)	28
22. Future Enhancements	29
23. Conclusion	30

1. Executive Summary

CredVault is a CRED-inspired credit card management platform that gives users a single, secure place to register their credit cards, track bills, and make payments. The original CredVault PRD (v1.0.0) specified a full enterprise microservices platform designed for a five-person team, spanning eleven services, an API gateway, event-driven messaging, and a dedicated frontend.

This document rescopes that vision into a Minimum Viable Product (MVP) that a single developer can realistically design, build, and test within 10–12 days, while still applying enterprise-grade backend engineering discipline: Clean Architecture, the Repository and Service patterns, dependency injection, JWT authentication, structured validation, centralized exception handling, and structured logging.

The MVP intentionally narrows scope to two backend services — Identity Service and Credit Management Service — and three functional modules: Card Management, Bill Management, and Payment Management. Every feature retained from the original vision is preserved in name, purpose, and domain terminology. Features that require a multi-developer team or infrastructure not practical for a solo 10–12 day build (API Gateway, RabbitMQ, Saga orchestration, Rewards, Notifications, Analytics, Admin/Support/Security/Audit services, Angular frontend, Docker/CI-CD) are documented in Section 22 as Future Enhancements rather than discarded.

Guiding Principle

Build a smaller, working version of the same platform — not a different platform. Every mandatory MVP feature must be demonstrably enterprise-grade in implementation, even where the surrounding infrastructure is simplified.

2. Project Vision

To build a production-quality fintech backend that demonstrates mastery of modern ASP.NET Core architecture and software engineering best practices — Clean Architecture, separation of concerns, secure authentication, and disciplined API design — within the realistic constraints of a single developer and a 10–12 day delivery window.

The MVP is designed as the architectural foundation of the original CredVault vision: a platform that can later absorb the API Gateway, event-driven communication, Saga-orchestrated payments, and additional domain services (Rewards, Notifications, Analytics, Security) described in v1.0.0, without requiring a rewrite of the core Identity and Credit Management services.

3. Problem Statement

Credit card users typically manage multiple cards across different banking apps, frequently miss due dates, and lack a consolidated view of their outstanding balances and payment history. CredVault addresses this by centralizing card, bill, and payment data behind a single, secure, authenticated backend API.

The MVP focuses on solving the foundational piece of this problem — reliable card registration, accurate bill generation, and safe payment processing — as the base layer on which reward tracking, spend analytics, and proactive notifications (all part of the original vision) can later be built.

4. Project Scope

4.1 In Scope (MVP)

- Identity Service: registration, login, JWT issuance, profile management, role management (User/Admin).
- Credit Management Service: Card, Bill, and Payment modules.
- Clean Architecture applied independently within each service.
- SQL Server persistence via EF Core 10, with a dedicated database per service.
- Swagger/OpenAPI documentation for all endpoints.
- Global exception handling, input validation, and structured logging.
- Optional Overview API aggregating a user's cards, bills, and payments (Section 16.5).

4.2 Out of Scope for MVP (see Section 22 — Future Enhancements)

- Ocelot API Gateway and centralized routing/rate limiting.
- RabbitMQ event-driven messaging between services.
- CQRS with MediatR and Saga orchestration (MassTransit) for payments.
- Rewards, Notification, Analytics, Support, Admin, Security, and Audit services.
- Angular 19 frontend / SPA.
- Redis caching, MFA/OTP, and KYC verification.
- Docker containerization, SonarQube, and CI/CD pipelines.

Deferring these items is a scope decision driven by team size and timeline, not a change of product vision. Section 22 documents how each maps back onto the MVP architecture.

5. Functional Scope

The table below distinguishes Mandatory MVP features (must be implemented within the 10–12 day window) from Optional Enhancements (nice to have if time permits, without expanding the core commitment).

Module	Feature	Classification
Identity	Registration, Login, JWT Authentication	Mandatory
Identity	Profile Management	Mandatory
Identity	Role Management (User / Admin)	Mandatory
Card	Add / Update / Delete / View Cards	Mandatory
Card	Luhn Validation & Issuer Detection	Mandatory
Card	Credit Limit & Outstanding Amount Tracking	Mandatory
Bill	Generate Bill, View Bills, Bill Details	Mandatory
Bill	Due Date, Minimum Due, Bill Status	Mandatory
Payment	Pay Bill, Payment History	Mandatory
Payment	Duplicate Payment Prevention	Mandatory
Payment	Auto Bill Status Update on Payment	Mandatory
Overview	Aggregated User Overview API	Optional

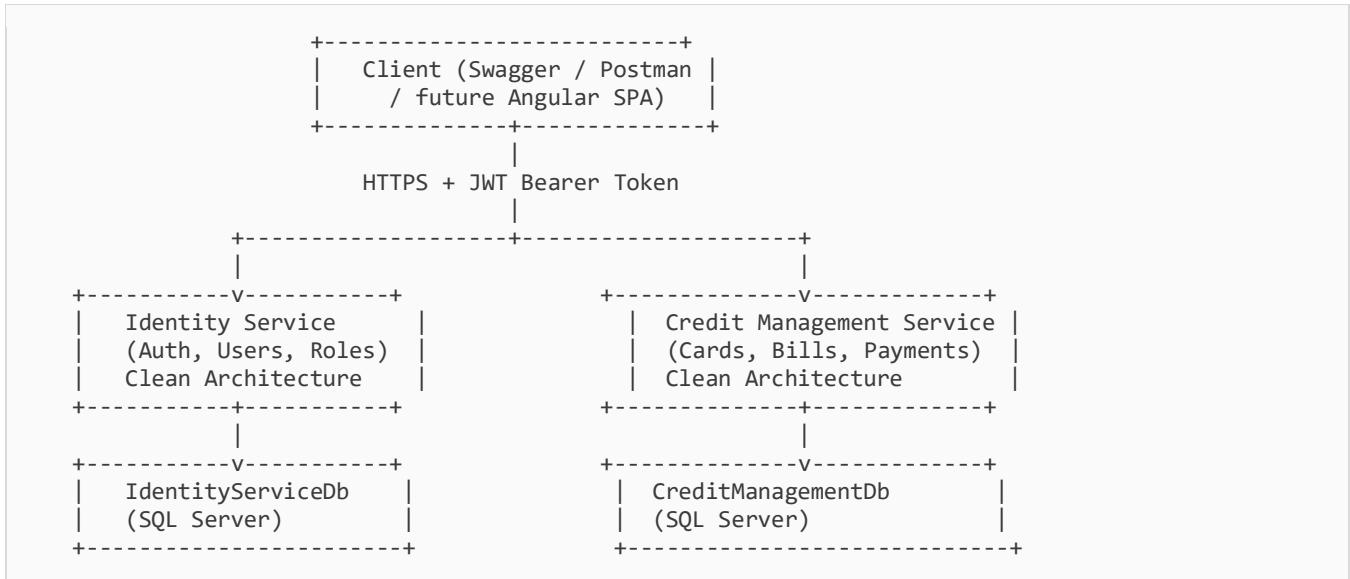
6. Technology Stack

Layer	Technology	Purpose
Backend Framework	ASP.NET Core Web API (.NET 10)	REST API host for both services
ORM	Entity Framework Core 10	Data access, migrations, change tracking
Database	SQL Server	Relational persistence, one database per service
Authentication	JWT Bearer Authentication	Stateless token-based auth shared across services
Architecture Style	Clean Architecture	Domain / Application / Infrastructure / API layering
Data Access Pattern	Repository Pattern	Abstracts EF Core behind interfaces
Business Logic Pattern	Service Pattern	Encapsulates use-case logic outside controllers
Dependency Injection	Built-in ASP.NET Core DI Container	Constructor injection across all layers
API Documentation	Swagger / OpenAPI (Swashbuckle)	Interactive API docs and contract-first testing
Validation	Data Annotations / FluentValidation	Request DTO validation
Exception Handling	Global Exception Middleware	Centralized, consistent error responses
Logging	Serilog	Structured logging to console and file sinks
Version Control	Git / GitHub	Source control and change history

Note: the original v1.0.0 stack (Angular 19, Ocelot Gateway, PostgreSQL/SQL Server, RabbitMQ, MassTransit, Redis, SonarQube, GitHub Actions, Serilog + Seq) remains the long-term target. The MVP stack above is a deliberate subset chosen for single-developer feasibility; Serilog is retained as-is, and SQL Server is standardized on for both services to simplify local setup.

7. High-Level Architecture

The MVP uses two independently deployable ASP.NET Core Web API services, each following Clean Architecture internally. There is no API Gateway in this phase — the client (Swagger UI, Postman, or a future frontend) calls each service directly over HTTPS. Both services trust the same JWT signing key, so a token issued by the Identity Service is accepted by the Credit Management Service without a shared session store.



Both services share the same JWT signing secret (configured via appsettings / user-secrets) so the Credit Management Service can validate tokens issued by the Identity Service without calling it synchronously. This mirrors the Identity Provider pattern described in the original v1.0.0 architecture, simplified to remove the API Gateway hop.

8. Microservice Overview

8.1 Identity Service

Attribute	Detail
Responsibilities	User Registration, User Login, JWT Authentication, User Profile Management, Role Management (User/Admin)
Database	IdentityServiceDb — Users, Roles
Suggested Port	5001
Owner (original v1.0.0)	Team Lead / Architect — retained as the developer's own responsibility in the MVP

8.2 Credit Management Service

Attribute	Detail
Responsibilities	Card Management, Bill Management, Payment Management
Database	CreditManagementDb — Cards, Bills, Payments
Suggested Port	5002
Owner (original v1.0.0)	Backend Dev (Card/Billing/Payment Services) — consolidated into one service for the MVP

Each service is independently deployable, owns its own database, and enforces the same four-layer Clean Architecture described in Section 9. This mirrors the per-service isolation principle from the original microservices design, just at a smaller service count.

9. Clean Architecture

Both services enforce the same four-layer Clean Architecture used in the original v1.0.0 design. Dependencies point inward: outer layers depend on inner layers, never the reverse. This keeps business rules independent of frameworks, databases, and delivery mechanisms.

Layer	Project	Contents
Domain	ServiceName.Domain	Entities, enums, domain constants, repository interfaces. No dependency on EF Core or ASP.NET Core.
Application	ServiceName.Application	DTOs, service interfaces and implementations, validators, mapping logic, business use cases.
Infrastructure	ServiceName.Infrastructure	EF Core DbContext, repository implementations, migrations, external integrations.
API (Presentation)	ServiceName.API	Controllers, middleware, Swagger configuration, DI composition root, JWT configuration.

Why it matters: business rules (e.g., Luhn validation, duplicate payment prevention) live in the Application layer, independent of whether the data eventually comes from SQL Server, a different database, or a message queue. This is the same rationale the original PRD gives for CQRS and DDD — this MVP keeps the layering discipline without adopting the full CQRS/MediatR machinery, which is deferred to Section 22.

10. Repository Pattern

Each entity (User, Card, Bill, Payment) is accessed through a repository interface defined in the Domain layer and implemented in the Infrastructure layer using EF Core. Controllers and Application services never reference DbContext or DbSet directly.

```
public interface ICardRepository
{
    Task<Card?> GetByIdAsync(Guid id);
    Task<IEnumerable<Card>> GetByUserIdAsync(Guid userId);
    Task AddAsync(Card card);
    void Update(Card card);
    void Delete(Card card);
}
```

Purpose: the Repository Pattern decouples the Domain and Application layers from EF Core specifics, making business logic unit-testable against an in-memory or mocked repository and making a future database or ORM change a localized, Infrastructure-only concern.

11. Service Pattern

Application services (e.g., CardService, BillService, PaymentService, AuthService) sit between controllers and repositories. They orchestrate business rules — validation, cross-entity checks, status transitions — while controllers remain thin and only handle HTTP concerns (model binding, status codes).

```
public interface IPaymentService
{
    Task<PaymentResponseDto> PayBillAsync(Guid userId, PayBillRequestDto request);
    Task<IEnumerable<PaymentResponseDto>> GetHistoryAsync(Guid userId);
}
```

Purpose: the Service Pattern keeps controllers focused on transport concerns and gives business logic a single, reusable, testable home — the same separation of concerns the original PRD achieves via MediatR command/query handlers, without requiring the CQRS pipeline for an MVP of this size.

12. Folder Structure

The solution is organized as two independent Clean Architecture solutions inside a single repository, mirroring the original microservice folder convention.

```
CredVault/
|
+-- IdentityService/
|   +-- Identity.API/
|   |   +-- Controllers/           (AuthController, UsersController)
|   |   +-- Middleware/           (ExceptionHandlingMiddleware)
|   |   +-- Program.cs
|   |   +-- appsettings.json
|   +-- Identity.Application/
|   |   +-- DTOs/
|   |   +-- Interfaces/
|   |   +-- Services/
|   |   +-- Validators/
|   +-- Identity.Domain/
|   |   +-- Entities/             (User, Role)
|   |   +-- Enums/
|   +-- Identity.Infrastructure/
|   |   +-- Persistence/          (IdentityDbContext, Migrations)
|   |   +-- Repositories/
|
+-- CreditManagementService/
|   +-- CreditManagement.API/
|   |   +-- Controllers/          (CardsController, BillsController,
|   |   |                           PaymentsController, OverviewController)
|   |   +-- Middleware/
|   |   +-- Program.cs
|   |   +-- appsettings.json
|   +-- CreditManagement.Application/
|   |   +-- DTOs/
|   |   +-- Interfaces/
|   |   +-- Services/             (CardService, BillService, PaymentService)
|   |   +-- Validators/
|   +-- CreditManagement.Domain/
|   |   +-- Entities/             (Card, Bill, Payment)
|   |   +-- Enums/               (BillStatus, PaymentStatus)
|   +-- CreditManagement.Infrastructure/
|   |   +-- Persistence/          (CreditManagementDbContext, Migrations)
|   |   +-- Repositories/
```


13. Database Design

Each service owns a dedicated SQL Server database, consistent with the original per-service database ownership principle. There is no cross-database foreign key between the two databases — UserId in the Credit Management database is a soft reference validated via JWT claims, not a SQL-enforced constraint.

13.1 IdentityServiceDb

Roles

Column	Type	Constraint
Id	UNIQUEIDENTIFIER	PK, default newid()
Name	NVARCHAR(50)	Unique, Not Null (e.g., 'User', 'Admin')

Users

Column	Type	Constraint
Id	UNIQUEIDENTIFIER	PK, default newid()
FullName	NVARCHAR(150)	Not Null
Email	NVARCHAR(200)	Unique, Not Null
PasswordHash	NVARCHAR(MAX)	Not Null (BCrypt hash)
PhoneNumber	NVARCHAR(20)	Nullable
RoleId	UNIQUEIDENTIFIER	FK -> Roles.Id, Not Null
IsActive	BIT	Not Null, default 1
CreatedAt	DATETIME2	Not Null, default getutcdate()
UpdatedAt	DATETIME2	Nullable

13.2 CreditManagementDb

Cards

Column	Type	Constraint
Id	UNIQUEIDENTIFIER	PK, default newid()
UserId	UNIQUEIDENTIFIER	Not Null (soft reference to Identity Service User)
CardHolderName	NVARCHAR(150)	Not Null
CardNumberMasked	NVARCHAR(25)	Not Null (e.g., **** * 1234)
CardNumberHash	NVARCHAR(MAX)	Not Null, unique (for duplicate-card detection)

Column	Type	Constraint
ExpiryMonth	INT	Not Null, Check 1-12
ExpiryYear	INT	Not Null
Issuer	NVARCHAR(50)	Not Null (Visa, Mastercard, RuPay, Amex — auto-detected)
CreditLimit	DECIMAL(18,2)	Not Null, Check > 0
OutstandingAmount	DECIMAL(18,2)	Not Null, default 0, Check >= 0
CreatedAt	DATETIME2	Not Null, default getutcdate()

Bills

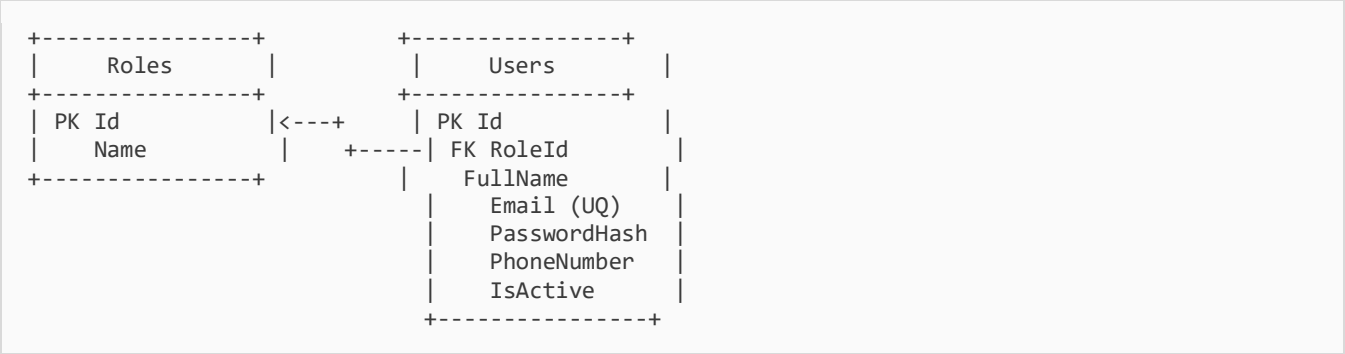
Column	Type	Constraint
Id	UNIQUEIDENTIFIER	PK, default newid()
CardId	UNIQUEIDENTIFIER	FK -> Cards.Id, Not Null, On Delete Cascade
BillingCycleStart	DATE	Not Null
BillingCycleEnd	DATE	Not Null
TotalAmount	DECIMAL(18,2)	Not Null, Check >= 0
MinimumDue	DECIMAL(18,2)	Not Null, Check >= 0
DueDate	DATE	Not Null
Status	NVARCHAR(20)	Not Null, default 'Unpaid' (Unpaid / Paid / Overdue)
GeneratedAt	DATETIME2	Not Null, default getutcdate()

Payments

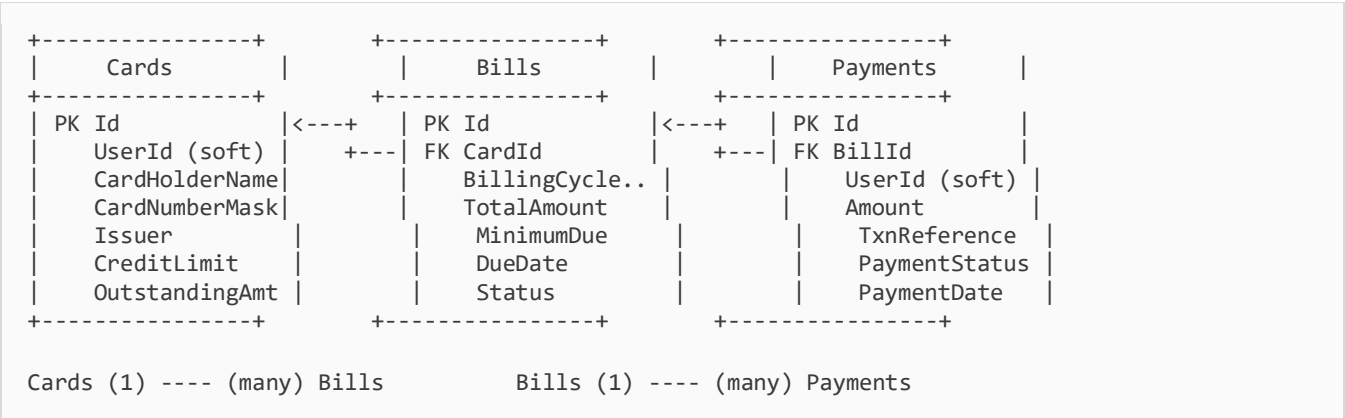
Column	Type	Constraint
Id	UNIQUEIDENTIFIER	PK, default newid()
BillId	UNIQUEIDENTIFIER	FK -> Bills.Id, Not Null
UserId	UNIQUEIDENTIFIER	Not Null (soft reference to Identity Service User)
Amount	DECIMAL(18,2)	Not Null, Check > 0
TransactionReference	NVARCHAR(100)	Unique, Not Null (idempotency / duplicate-payment guard)
PaymentStatus	NVARCHAR(20)	Not Null (Success / Failed)
PaymentDate	DATETIME2	Not Null, default getutcdate()

14. Entity Relationship Diagram

14.1 IdentityServiceDb



14.2 CreditManagementDb



15. Functional Requirements

15.1 Identity Module

#	Feature	Description	Priority
FR-1.1	User Registration	Register with full name, email, password. Validate email format, enforce password strength. Hash password with BCrypt before persisting.	Mandatory
FR-1.2	Login	Authenticate with email + password. Return a signed JWT access token containing UserId, Email, and Role claims.	Mandatory
FR-1.3	JWT Authentication	All Credit Management endpoints require a valid Bearer token. Token includes UserId claim used to scope card/bill/payment queries.	Mandatory
FR-1.4	Profile Management	Authenticated user can view and update their own profile (name, phone number).	Mandatory
FR-1.5	Role Management	Two roles: User (default) and Admin. Roles seeded at startup; Admin role reserved for future administrative endpoints.	Mandatory

15.2 Card Module

#	Feature	Description	Priority
FR-2.1	Add Card	Register a new card with card number, holder name, expiry, and credit limit. Card number validated with the Luhn algorithm before storage.	Mandatory
FR-2.2	Luhn Validation	Reject card numbers that fail the Luhn checksum with a 400 response before any persistence occurs.	Mandatory
FR-2.3	Card Issuer Detection	Detect issuer (Visa, Mastercard, RuPay, Amex) from the card number's IIN/BIN prefix and store it alongside the card.	Mandatory
FR-2.4	Update / Delete Card	Owner can update editable fields (holder name, credit limit) or remove a card they own.	Mandatory
FR-2.5	View Cards / Card Details	List all cards for the authenticated user (masked card numbers only) and fetch full details for one card.	Mandatory
FR-2.6	Credit Limit & Outstanding Amount	Track credit limit at add-time and running outstanding amount, updated as bills are generated and paid.	Mandatory

15.3 Bill Module

#	Feature	Description	Priority
FR-3.1	Generate Bill	Create a bill for a card for a given billing cycle, computing total amount and minimum due (e.g., 5% of total, configurable).	Mandatory

#	Feature	Description	Priority
FR-3.2	Due Date	Each bill carries a due date, defaulted to a fixed number of days after cycle end (e.g., 18 days).	Mandatory
FR-3.3	View Bills / Bill Details	List all bills for the authenticated user's cards and fetch full details for a single bill.	Mandatory
FR-3.4	Bill Status	Bill status is Unpaid, Paid, or Overdue. Status transitions automatically when a payment succeeds or the due date passes.	Mandatory

15.4 Payment Module

#	Feature	Description	Priority
FR-4.1	Pay Bill	Submit a payment against an unpaid bill for an amount up to the outstanding total.	Mandatory
FR-4.2	Prevent Duplicate Payments	A unique, client-supplied transaction reference is enforced at the database level so a retried request cannot create a second payment.	Mandatory
FR-4.3	Payment History	List all payments made by the authenticated user, most recent first.	Mandatory
FR-4.4	Auto-Update Bill Status	On successful payment covering the full outstanding amount, the related bill's status is automatically set to Paid.	Mandatory

15.5 Optional Enhancement — Overview

#	Feature	Description	Priority
FR-5.1	User Overview API	GET /api/users/me/overview aggregates user info, registered cards, bills, pending bills, and recent payments into a single JSON response.	Optional

16. API Design

All endpoints below (except Register and Login) require a valid JWT Bearer token. Routes on the Credit Management Service resolve the authenticated user from the token's UserId claim — a user can only ever see and modify their own cards, bills, and payments.

16.1 Authentication & Users — Identity Service

Method	Route	Purpose	Request DTO	Response DTO	Status Codes
POST	/api/auth/register	Register a new user	RegisterRequestDto	AuthResponseDto	201, 400, 409
POST	/api/auth/login	Authenticate and issue JWT	LoginRequestDto	AuthResponseDto	200, 400, 401
GET	/api/users/me	Get current user profile	—	UserProfileDto	200, 401
PUT	/api/users/me	Update current user profile	UpdateProfileRequestDto	UserProfileDto	200, 400, 401

16.2 Cards — Credit Management Service

Method	Route	Purpose	Request DTO	Response DTO	Status Codes
POST	/api/cards	Add a new card	AddCardRequestDto	CardResponseDto	201, 400, 401
GET	/api/cards	List the user's cards	—	List<CardResponseDto>	200, 401
GET	/api/cards/{id}	Get card details	—	CardDetailsDto	200, 401, 404
PUT	/api/cards/{id}	Update a card	UpdateCardRequestDto	CardResponseDto	200, 400, 401, 404
DELETE	/api/cards/{id}	Delete a card	—	—	204, 401, 404

16.3 Bills — Credit Management Service

Method	Route	Purpose	Request DTO	Response DTO	Status Codes
POST	/api/cards/{cardId}/bills	Generate a bill for a card	GenerateBillRequestDto	BillResponseDto	201, 400, 401, 404
GET	/api/bills	List the user's bills	—	List<BillResponseDto>	200, 401
GET	/api/bills/{id}	Get bill details	—	BillDetailsDto	200, 401, 404

16.4 Payments — Credit Management Service

Method	Route	Purpose	Request DTO	Response DTO	Status Codes
POST	/api/payments	Pay a bill	PayBillRequestDto	PaymentResponseDto	201, 400, 401, 404, 409
GET	/api/payments	List payment history	—	List<PaymentResponseDto>	200, 401
GET	/api/payments/{id}	Get payment details	—	PaymentDetailsDto	200, 401, 404

16.5 Optional Enhancement — Overview

Optional — Not Part of Mandatory MVP

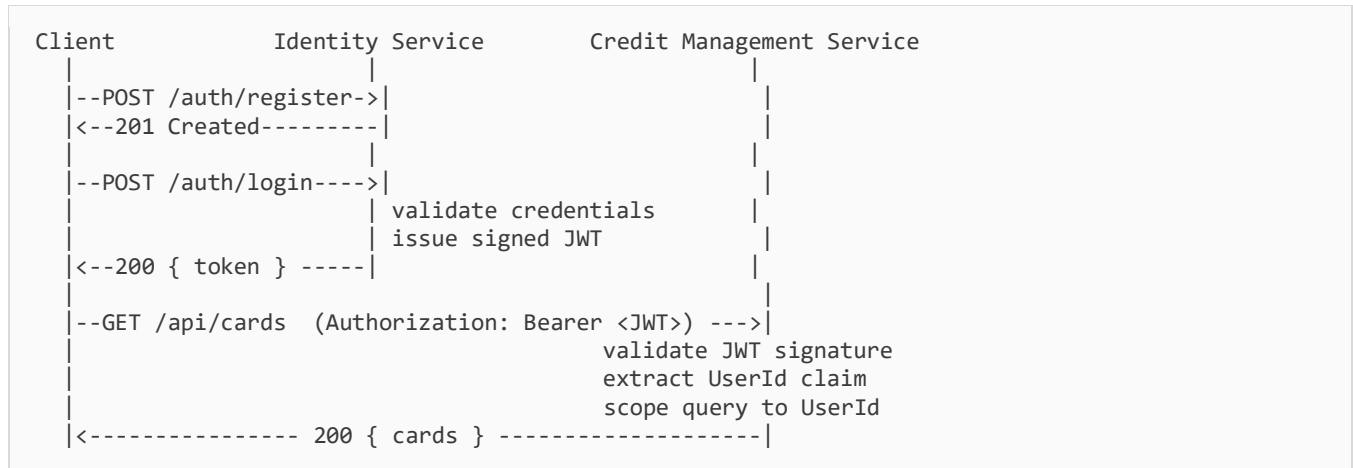
This endpoint is an optional enhancement to be implemented only if the mandatory scope is complete ahead of schedule.

Method	Route	Purpose	Request DTO	Response DTO	Status Codes
GET	/api/users/me/overview	Aggregated dashboard for the user	—	UserOverviewDto	200, 401

UserOverviewDto composes: user info, registered cards, all bills, pending (unpaid/overdue) bills, and the five most recent payments — a single call replacing four separate round trips.

17. Authentication Flow

Both services validate the same JWT using a shared signing key configured via appsettings/user-secrets. The Identity Service is the only service that issues tokens; the Credit Management Service only validates them.



Token contents: UserId, Email, and Role claims, signed with an HMAC-SHA256 (or RS256) key, with a practical expiry (e.g., 60 minutes) suitable for an MVP without a refresh-token flow. Refresh tokens, OTP/MFA, and token blacklisting are deferred to Future Enhancements (Section 22) as part of the original Multi-Factor Auth and Device/Session Management requirements.

18. Validation Strategy

- Every request DTO is decorated with Data Annotations (or validated via FluentValidation) — required fields, string lengths, ranges, and formats are enforced before a controller action runs.
- A global [ApiController] model-state check automatically returns 400 Bad Request with a structured error payload when annotation validation fails.
- Domain-specific business rules — Luhn checksum, issuer detection, credit-limit and outstanding-amount bounds, duplicate transaction reference — are enforced in the Application/Service layer, not in controllers.
- Payment amount is validated against the bill's remaining outstanding amount before a payment is accepted.
- Validation failures are surfaced as errors, never silently corrected, consistent with the same DTO-level validation approach used across all services in v1.0.0.

19. Exception Handling Strategy

Both services register a global exception-handling middleware so controllers never contain try/catch blocks for expected error conditions.

```
app.UseExceptionHandler(errorApp =>
{
    errorApp.Run(async context =>
    {
        var feature = context.Features.Get<IExceptionHandlerFeature>();
        var (status, title) = feature.Error switch
        {
            NotFoundException      => (404, "Resource Not Found"),
            ValidationException      => (400, "Validation Failed"),
            ConflictException        => (409, "Conflict"),
            UnauthorizedException    => (401, "Unauthorized"),
            _                       => (500, "Internal Server Error")
        };
        context.Response.StatusCode = status;
        await context.Response.WriteAsJsonAsync(new ProblemDetails
        {
            Title = title, Status = status, Detail = feature.Error.Message
        });
    });
});
```

- Custom exception types: NotFoundException, ValidationException, ConflictException, UnauthorizedException — thrown from the Application layer, mapped centrally to HTTP status codes.
- Every error response follows the same ProblemDetails-shaped JSON contract, regardless of which endpoint or service produced it.
- Unhandled exceptions are caught, logged with full context (Section 20), and never leak stack traces to the client in a non-development environment.

20. Logging Strategy

Both services use Serilog for structured logging, matching the original v1.0.0 choice of Serilog for structured, queryable logs — scoped down to console and rolling-file sinks for the MVP (Seq/centralized log aggregation is deferred to Section 22).

- A logging middleware records method, route, status code, and duration for every request.
- Each request is tagged with a correlation ID (generated per request) included in every log line for that request, so a single call's log trail can be traced even without distributed tracing.
- Log levels: Information for normal request/response flow, Warning for validation failures and business-rule rejections (e.g., duplicate payment), Error for unhandled exceptions.
- Sensitive data (passwords, full card numbers, JWTs) is never written to logs — only masked card numbers and user IDs.

21. Development Timeline (10–12 Days)

Sequenced so Identity Service is fully functional before Credit Management Service depends on it for authentication, and so each module is complete (Domain → Infrastructure → API) before moving to the next.

Day	Focus	Deliverables
1	Project Setup	Repository, solution structure for both services, SQL Server instance, EF Core project references, base Clean Architecture skeleton.
2	Identity — Domain & Application	User and Role entities, DTOs (Register/Login/Profile), service interfaces, validators.
3	Identity — Infrastructure & API	EF Core DbContext + migrations, repository implementation, Register/Login endpoints, JWT issuance.
4	Identity — Completion	Profile management endpoints, role seeding, Swagger, manual testing of the full Identity flow.
5	Credit Mgmt — Domain & Application (Card)	Card entity, Luhn validation, issuer detection logic, Card DTOs and service interfaces.
6	Card Module — Infrastructure & API	EF Core setup for CreditManagementDb, Card repository, full Card CRUD endpoints.
7	Bill Module	Bill entity, bill generation logic (total, minimum due, due date), Bill DTOs, repository, and endpoints.
8	Payment Module	Payment entity, duplicate-payment prevention via unique transaction reference, payment endpoints, automatic bill status update.
9	Cross-Cutting Concerns	Global exception middleware, validation pipeline, Serilog logging wired into both services.
10	Cross-Service Auth Integration	Shared JWT signing key wiring, end-to-end test of Identity token accepted by Credit Management Service, optional Overview API.
11	Hardening	Bug fixing, Swagger polish, seed/demo data, README and setup documentation.
12	Buffer	Final regression pass, edge-case testing (duplicate payments, invalid Luhn numbers, expired bills), deployment readiness check.

22. Future Enhancements

These items are preserved from the original v1.0.0 enterprise architecture. They are not part of the MVP's mandatory scope, but the Clean Architecture and service boundaries established in the MVP are designed to accommodate them without a rewrite.

Enhancement	Original v1.0.0 Reference	Why Deferred
API Gateway	Ocelot Gateway — routing, rate limiting, JWT validation	Adds an extra deployable component and routing config not essential while there are only two services.
Event-Driven Messaging	RabbitMQ across all 11 services	Requires broker infrastructure and async consumer code disproportionate to a 2-service MVP.
CQRS + MediatR	Commands/Queries per service via MediatR	Read/write separation benefits scale with team size and service count more than solo development.
Saga Orchestration	MassTransit State Machine for payment workflow	Distributed-transaction coordination is only necessary once payments span multiple independent services.
Rewards Service	Points, tiers, redemption, expiry	A net-new domain service; layered on top of Payment data once the core payment flow is stable.
Notification Service	RabbitMQ-triggered email via SMTP/SendGrid	Depends on event-driven messaging being in place first.
Analytics Service	Spending reports, health score, charts	Consumes historical Card/Bill/Payment data that the MVP now makes available for future aggregation.
Admin / Support Services	Internal dashboard, ticketing, fraud/user management	Operational tooling, valuable once there are real users to support.
Security / Audit Services	Fraud detection, risk scoring, compliance logs	Builds on the correlation-ID logging already present in the MVP.
MFA / OTP & KYC	TOTP/email OTP, Aadhaar/PAN verification	Compliance-grade identity features layered onto the existing Identity Service and Role model.
Angular 19 Frontend	SPA — Auth, Dashboard, Cards, Bills, Rewards, Analytics	Consumes the REST API designed in Section 16 as-is; no backend contract changes required.
Docker + CI/CD + SonarQube	Containerization, GitHub Actions, quality gates	Operational maturity step once the codebase and test suite are established.

23. Conclusion

This MVP rescope keeps CredVault's identity, domain, and long-term architectural vision intact while making the build achievable for one developer in 10–12 days. The Identity and Credit Management services, built with Clean Architecture, the Repository and Service patterns, JWT authentication, centralized validation, exception handling, and structured logging, form a genuine enterprise-grade foundation — not a simplified toy version of the product.

Every capability deferred in Section 22 has a clear landing spot in the existing architecture, so CredVault can grow from this MVP toward the full v1.0.0 vision — API Gateway, event-driven microservices, Saga-orchestrated payments, and a dedicated frontend — without re-architecting the core.

End of Document

CredVault Product Requirements Document v2.0.0 — MVP Rescope Edition. Confidential — For Internal Use Only.